



Project Title	Rising Waters: Machine Learning Based Flood Prediction System		
Developer	S. Ramya Sathwika	Team ID	[Team ID]
Date	26 June	Phase / Document	Brainstorming & Ideation

2. Define Problem Statement

Before any architecture or model choices were made, the problem was deliberately framed from the perspective of the end user rather than the technology. The “Empathize Canvas” format below forces the problem to be stated in terms of who is affected, what they are trying to do, and what is currently stopping them — which keeps the later technical decisions (which features to use, how to present the output) anchored to a real need rather than to whatever the dataset happens to contain.

2.1 Problem Statement (Empathize Canvas Format)

I am (User)	I'm trying to	But	Because	Which makes me feel
A resident / local authority in a floodprone region	Understand the likelihood of an upcoming flood so I can prepare or act in time	I only have access to generalized regional advisories, not a quick localized assessment	There is no accessible tool that converts available weather data into an immediate, understandable risk level	Uncertain, under-prepared, and reactive rather than proactive

2.2 Formal Problem Statement

Residents and local disaster-management stakeholders in flood-prone areas lack a simple, data-driven tool that converts available meteorological data (temperature, humidity, cloud cover, seasonal and annual rainfall) into an immediate, interpretable flood-risk classification, along with actionable guidance, to support early preparedness and response.

This statement deliberately combines two requirements that are often treated separately in academic ML projects: producing an accurate prediction, and presenting that prediction in a form a non-technical user can act on. A model that outputs “0.62” is not, by itself, a solution to this problem — the solution only exists once that number is translated into a severity tier and a concrete recommendation, which is why the insight-generation and protocol layers in app.py are treated as first-class parts of the solution rather than cosmetic additions.

